Microservices Communication in Spring Boot

Focus

Create two microservices and make them communicate with each other.

Learning Objectives

After completing this topic, you will be able to:

- Understand what Microservices are.
- Understand the concept of Service Boundaries.
- Design Customer Service and Order Service.
- Learn how microservices communicate using REST APIs.
- Understand why communication between services is required.

What are Microservices?

A **Microservice** is a small, independent application that focuses on performing a single business function. Instead of building one large application that handles everything, we divide the application into multiple small services.

Each microservice:

- Performs one specific business responsibility.
- Can be developed independently.
- Can be deployed independently.
- Can have its own database.
- Communicates with other services using APIs.

Example

Why Microservices?

Suppose an online shopping application contains everything inside one project.

```
Shopping Application

├── Customer Module
├── Order Module
├── Product Module
├── Payment Module
├── Shipping Module
├── Notification Module
```

As the application grows:

- The project becomes very large.
- Deployment becomes difficult.
- One small change requires redeploying the entire application.

Service Boundaries

What is a Service Boundary?

A **Service Boundary** defines the responsibilities of a microservice.

It answers the question:

> **"What should belong inside this service?"**

A good service boundary ensures that every microservice has only one responsibility.

This follows the **Single Responsibility Principle (SRP)**.

Poor Service Boundary

Suppose we create one service called Customer Service.

```
Customer Service

├── Customer Registration
├── Customer Login
├── Customer Orders
├── Product Management
├── Payment Processing
├── Shipping
└── Inventory
```

This service is trying to do everything.

Problems:

- Difficult to maintain
- Difficult to test

Good Service Boundary

Instead, split the application based on business responsibilities.

```
Customer Service

├── Customer Registration
├── Customer Login
├── Customer Profile
└── Customer Details
```

```
Order Service

├── Create Order
├── Cancel Order
├── Order History
└── Order Status
```

```
Product Service
```

Customer Service Design

Purpose

Customer Service is responsible for managing customer-related information.

It should not contain order logic or payment logic.

Responsibilities

Customer Service performs operations such as:

- Register Customer
- View Customer Details
- Update Customer Information
- Delete Customer
- Search Customer

Database

Customer Service owns its own database.

Example:

```
customer_db
```

Customer Table

Column	Description
id	Customer ID
name	Customer Name
email	Email Address
phone	Phone Number

REST APIs

Create Customer

```
POST /customers
```

Request

```
{
  "name": "John",
  "email": "john@gmail.com",
  "phone": "9876543210"
}
```

Response

```
{
  "id": 101,
  "message": "Customer Created Successfully"
}
```

Get Customer

```
GET /customers/101
```

Response

```
{
  "id": 101,
  "name": "John",
  "email": "john@gmail.com",
  "phone": "9876543210"
}
```

Update Customer

```
PUT /customers/101
```

Delete Customer

```
DELETE /customers/101
```

Project Structure

```
customer-service

src
└── main
    ├── java
    │   ├── controller
    │   ├── service
    │   ├── repository
    │   ├── entity
    │   └── dto
    │
    └── resources
        └── application.properties
```

Order Service Design

Purpose

Order Service manages customer orders.

It should only contain order-related business logic.

Responsibilities

- Create Order
- View Order
- Update Order Status
- Cancel Order
- Order History

Database

Order Service owns its own database.

Example:

```
order_db
```

Order Table

Column	Description
id	Order ID
customerId	Customer ID
product	Product Name
quantity	Quantity
price	Price

REST APIs

Create Order

```
POST /orders
```

Request

```
{
  "customerId": 101,
  "product": "Laptop",
  "quantity": 1,
  "price": 65000
}
```

Response

```
{
  "orderId": 5001,
```

Get Order

```
GET /orders/5001
```

Get Orders of a Customer

```
GET /orders/customer/101
```

Cancel Order

```
DELETE /orders/5001
```

Project Structure

```
order-service

src
└── main
    ├── java
    │   ├── controller
    │   ├── service
    │   ├── repository
    │   ├── entity
    │   ├── dto
    │   └── client
    │
    └── resources
        └── application.properties
```

Why Should Order Service Communicate with Customer Service?

Suppose a customer places a new order.

Before creating the order, Order Service must verify whether the customer actually exists.

Instead of storing duplicate customer information, Order Service sends an HTTP request to Customer Service.

Communication flow:

```
Customer places Order
        |
        ▼
Order Service
        |
```

REST-Based Communication

Microservices commonly communicate using **REST APIs**.

REST stands for **Representational State Transfer**.

Communication happens over the HTTP protocol.

Most REST APIs exchange data in **JSON** format.

Communication Flow

```
Order Service

      |

HTTP GET Request

      |

      ▼

Customer Service

GET /customers/101

      |

Returns JSON

      ▼

{
   "id":101,
   "name":"John",
   "email":"john@gmail.com"
}
```

HTTP Methods Used in REST

GET

Used to retrieve data.

Example:

```
GET /customers/101
```

Returns customer details.

POST

Used to create new data.

Example:

```
POST /orders
```

Creates a new order.

PUT

Used to update existing data.

Example:

```
PUT /customers/101
```

Updates customer information.

DELETE

Used to remove data.

Example:

```
DELETE /orders/5001
```

Deletes or cancels an order.

JSON Request and Response

Request

```
{
  "customerId": 101,
  "product": "Laptop",
  "quantity": 1,
  "price": 65000
}
```

Response

```
{
  "orderId": 5001,
  "status": "CREATED"
}
```

JSON is lightweight, human-readable, and widely used for communication between

Summary

In this part, you learned:

- What Microservices are.
- Why Microservices are used.
- The importance of Service Boundaries.
- How Customer Service is designed.
- How Order Service is designed.
- Why Order Service communicates with Customer Service.
- How REST-based communication works.
- Common HTTP methods used in microservices.
- JSON request and response format.

Next: Part 2 covers:

- RestClient
- OpenFeign
- Comparing RestClient vs OpenFeign
- Complete Communication Flow
- Handling Service Failures (Timeout, Retry, Circuit Breaker, Fallback)